FinalProject - Graphical and User Interfaces

Luca David Șandru

May 2026

1 Introduction

This project presents an interactive 3D Robot Interaction Arena implemented using OpenGL and FreeGLUT. The scene contains three autonomous robots moving continuously on curved trajectories over a mathematically generated terrain surface.

2 Scene Design Decisions

Several design decisions were made in order to create a clear and visually interactive 3D environment.

2.1 Robot Design

The robots were designed using a modular hierarchical structure:

- torso,
- head,
- upper/lower arms,
- upper/lower legs,
- articulated joints.

Each body part was implemented separately using scaled OpenGL primitive objects such as cubes and spheres. This allowed independent transformations and animations for every body part.

2.2 Terrain Design

Instead of using a flat plane, the scene uses a mathematically generated curved terrain surface (the green surface). This improves scene realism and demonstrates parametric surface rendering.

The terrain was designed with:

- smooth sinusoidal function, using height variation,
- continuous normals for lighting,
- procedural checkerboard texture mapping.

2.3 Path Design

Each robot follows a different elliptical path around the center of the arena.

The paths were intentionally designed with:

- different radii,
- phase offsets,
- different movement speeds,

in order to naturally create robot interactions during the simulation.

3 Transformation Hierarchy

The robots were implemented using hierarchical modeling based on OpenGL matrix stack operations.

3.1 Hierarchical Structure

The torso acts as the root transformation node. All other body parts inherit transformations from the torso.

The hierarchy is organized as follows:

- Torso
  - Head
  - Left Arm
    - Upper Arm
    - Forearm
  - Right Arm
    - Upper Arm
    - Forearm
  - Left Leg

    - Upper Leg
    - Lower Leg
  - Right Leg
    - Upper Leg
    - Lower Leg

3.2 Matrix Stack Usage

The implementation heavily uses:

```
glPushMatrix()
glPopMatrix()
glTranslatef()
glRotatef()
glScalef()
```

Each body component is transformed relative to its parent component. This hierarchical structure allows:

- realistic walking movement,
- waving gestures,
- reusable transformation logic.

4 Curved Surface Generation

The terrain surface is generated procedurally using a mathematical height function:

$$y = 0.5 \sin(0.4x) + 0.3 \cos(0.5z) \tag{1}$$

This creates a smooth continuous terrain with hills and valleys.

4.1 Surface Tessellation

The terrain is discretized into a grid of samples and rendered using:

`GL_TRIANGLE_STRIP`

For each vertex:

- height is computed using the surface equation,
- normals are calculated numerically,
- texture coordinates are generated.

4.2 Surface Normals

Normals are approximated using finite differences between neighboring height values.

This allows:

- smooth shading,
- correct diffuse lighting,
- realistic specular highlights.

5 Curved Path Generation

Robot trajectories are generated procedurally using parametric equations.

For each robot:

$$x = a\cos(t) \tag{2}$$

$$z = b\sin(t) \tag{3}$$

Different radii and phase offsets are used for every robot.

This creates:

- continuous cyclic movement,
- smooth robot navigation,
- dynamic interaction opportunities.

5.1 Robot Orientation

Robot orientation is computed automatically from the tangent direction of the path using:

`atan2()`

This ensures that every robot continuously faces its movement direction.

6 Robot Animation and Interaction

6.1 Walking Animation

Walking is implemented using sinusoidal oscillations applied to:

- arms,
- upper legs,
- lower legs.

This creates periodic walking movement.

6.2 Robot Interactions

The robots interact dynamically when they approach each other.

Distance checks are performed in the XZ plane using Euclidean distance computations.

Three interaction states were implemented:

- Robot 0 greeting Robot 1,
- Robot 1 greeting Robot 2,
- Robot 0 greeting Robot 2.

When robots become sufficiently close:

- waving animation is activated,
- movement speed is modified,
- speech text appears above the robots.

6.3 Speech Rendering

Text rendering was implemented using:

```
glutBitmapCharacter()
```

Messages such as:

```
"Hi, Robot 1!"
```

appear dynamically above interacting robots.

7 Lighting and Materials

Two lighting systems were implemented.

7.1 Directional Lighting

The first mode simulates sunlight using a directional light source.

Characteristics:

- constant light direction,
- diffuse illumination,
- global ambient light.

7.2 Moving Point Light

The second mode uses a moving point light rotating around the scene.

This creates:

- dynamic highlights,
- varying shadows,
- stronger depth perception.

7.3 Material Types

Three material models were implemented:

- matte material,
- plastic material,
- shiny metallic material.

Different robot components use different material properties in order to improve realism.

8 Texture Mapping

Procedural textures were generated directly in code.

8.1 Terrain Texture

The terrain uses a green checkerboard texture generated algorithmically.

8.2 Robot Texture

The robot torso uses a red and white striped checker texture.

Texture mapping demonstrates:

- OpenGL texture coordinates,
- texture wrapping,
- texture filtering.

Textures can be enabled or disabled dynamically during execution.

9 Camera and Projection Systems

9.1 Camera Modes

Four camera modes were implemented:

- front view,
- side view,
- top view,
- orbit/free camera.

The orbit camera supports:

- mouse rotation,
- zooming,
- interactive scene exploration.

9.2 Projection Modes

Two projection systems were implemented:

- perspective projection,
- orthographic projection.

Perspective projection demonstrates:

- depth perception,
- scale variation,
- vanishing-point behavior.

Orthographic projection removes perspective distortion and preserves parallelism.

10 User Controls

The application supports interactive keyboard and mouse controls.

10.1 Keyboard Controls

- W – Start/stop animation
- + / - – Increase/decrease animation speed
- I – Enable/disable interactions
- L – Change lighting mode
- T – Toggle textures
- P – Perspective projection
- O – Orthographic projection
- 1 – Front camera
- 2 – Side camera
- 3 – Top camera
- 4 – Orbit camera
- R – Reset scene
- ESC – Exit application

10.2 Mouse Controls

- Left mouse drag – Rotate orbit camera
- Mouse wheel – Zoom in/out

11 Screenshots

11.1 Main Scene

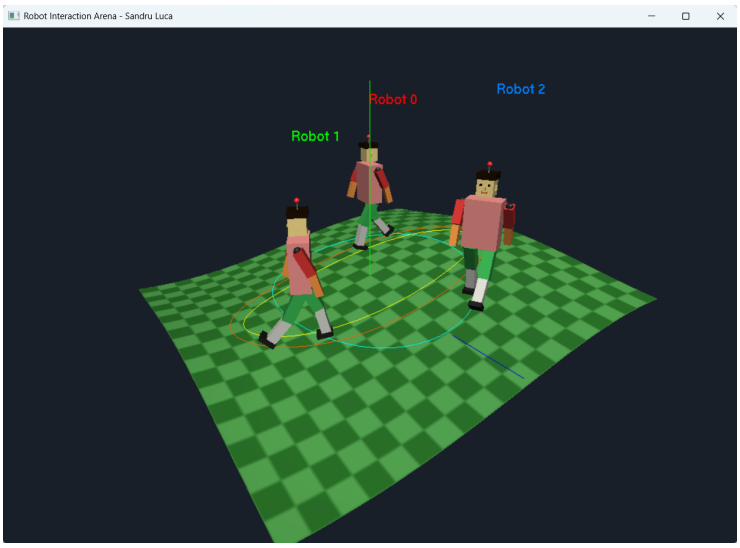

Figure 1: **Main Scene** - when opening the UI

11.2 Robot Interaction Example

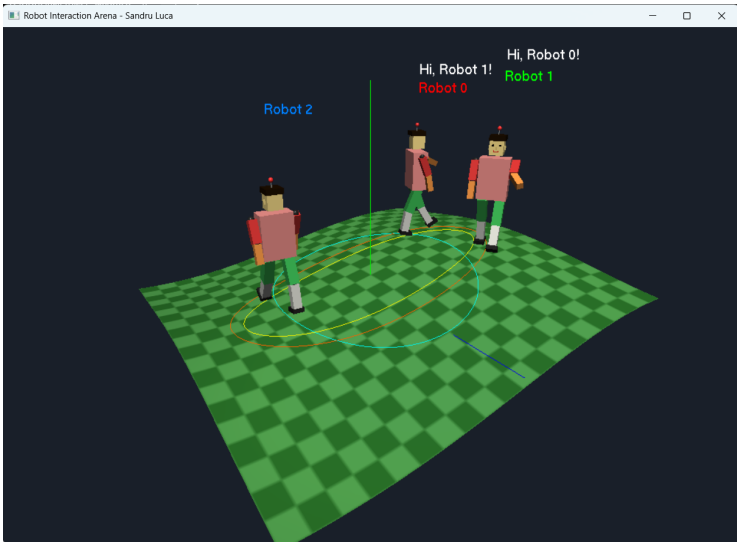

Figure 2: **Robot Interaction** between **Robot0** and **Robot1**

11.3 Perspective Projection

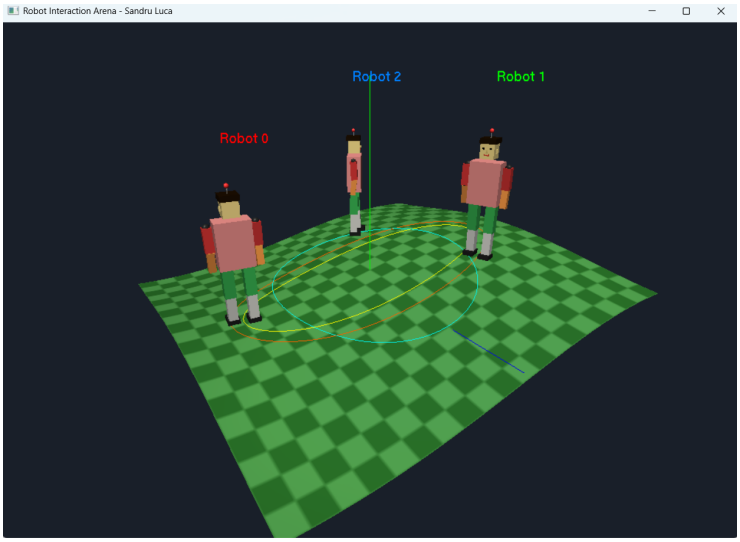

Figure 3: **Perspective Projection** - same as default

11.4 Orthographic Projection

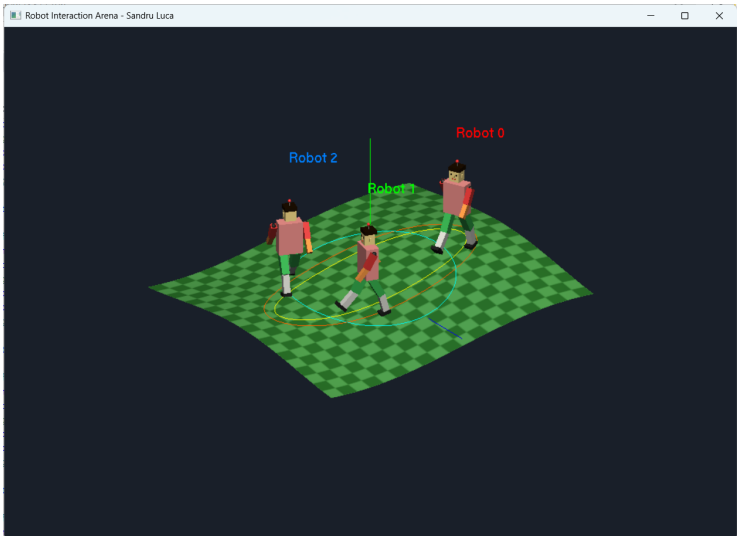

Figure 4: **Ortographic Projection** - the other projection

11.5 Lighting Mode 1

Figure 5: **Lighting Mode 1** - Same as default

11.6 Lighting Mode 2

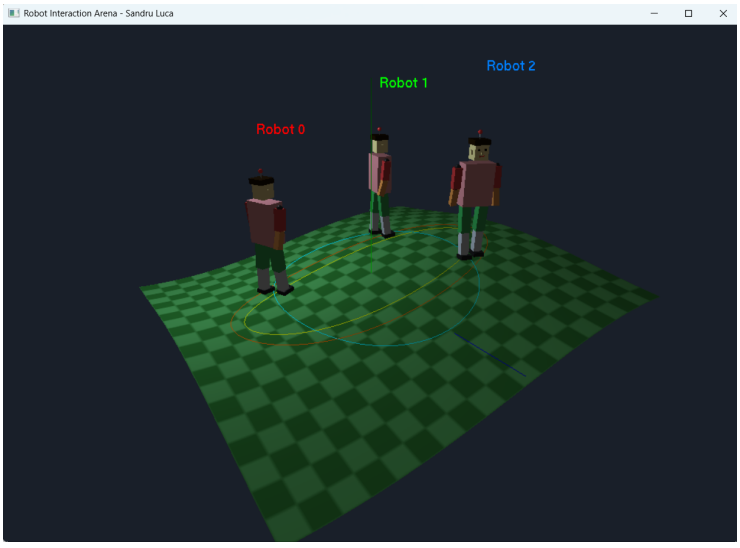

Figure 6: **Lighting Mode 2** - Darker as Lighting Mode 1

11.7 Textured Terrain and Robots

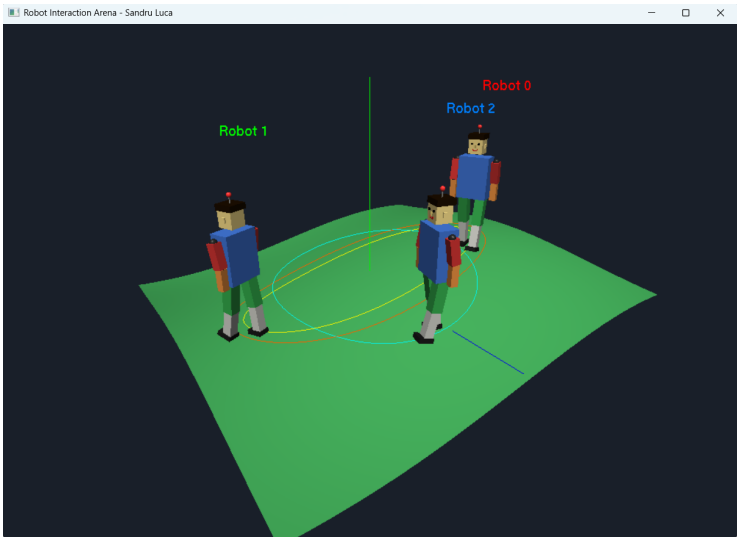

Figure 7: **Textured Terrain** - different surface and torso color of robots

12 Conclusion

The project successfully demonstrates advanced OpenGL rendering techniques including:

- hierarchical robot modeling,
- procedural surface generation,
- parametric animation,
- dynamic robot interactions,
- multiple lighting systems,
- texture mapping,
- camera and projection systems.

The final result is an interactive animated 3D environment containing multiple autonomous robots capable of movement, interaction, and visual communication.

For downloading, testing and more information, project can be accessed via **Github**: `https://github.com/LucaSandru/Robot-InteractionPlane`